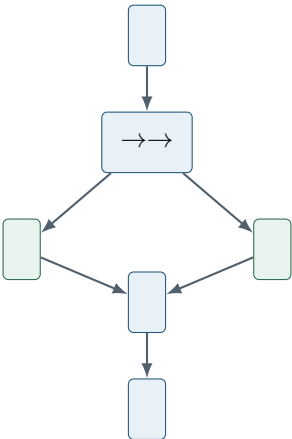



```
:synth ([a] -> [a]) where length result == length input
```

QueryEvidenceSemanticVerdict
seq

f50b5eac8672
0160237e4bdf
0bd679a404be

synthesis/src/Language/Haskell/Synthesis



Query.hs
Search.hs
Candidate.hs
Generated.hs
Selection.hs
Declaration.hs

```
data Expression local
  = Local local
  | Global Name
  | Lambda [Pattern local] (Expression local)
  | Apply (Expression local) (Expression local)
  | VisibleTypeApplication (Expression local) VisibleTypeArgument
  | Tuple [Expression local]
  | Hole local
  | Let (Pattern local) (Expression local) (Expression local)
  | Case (Expression local) [(Pattern local, Expression local)]
```

SearchNode

unsafePerformIO

Leant.Synth.EngineMain.hssorry

```
set_option autoImplicit true in
noncomputable example : (goal) := (candidate)
```

SynthOutcomeSynthCandidate

synth-funcheck-synth

```
data Contract ty variable = Contract
  { contractInputs      :: [Binder ty variable]
  , contractPrecondition :: Formula variable
  , contractResult      :: Binder ty variable
  , contractPostcondition :: Formula variable
  , contractDomain      :: SemanticDomain
  , contractMode        :: VerificationMode
  , contractObjectives  :: [Objective]
  }
```

```
data SemanticDomain
  = MathematicalTotal
  | FiniteSpineTotal
  | BoundedOperational Bounds
```

```
data VerificationMode
  = RequireProof
  | AcceptSolverUnsat
  | AcceptBoundedValidation Bounds
```

f

$$\forall x. P(x) \Rightarrow Q(x, f(x)).$$

$$\exists x. P(x) \wedge \neg Q(x, f(x)).$$

sat

unsat

unknown

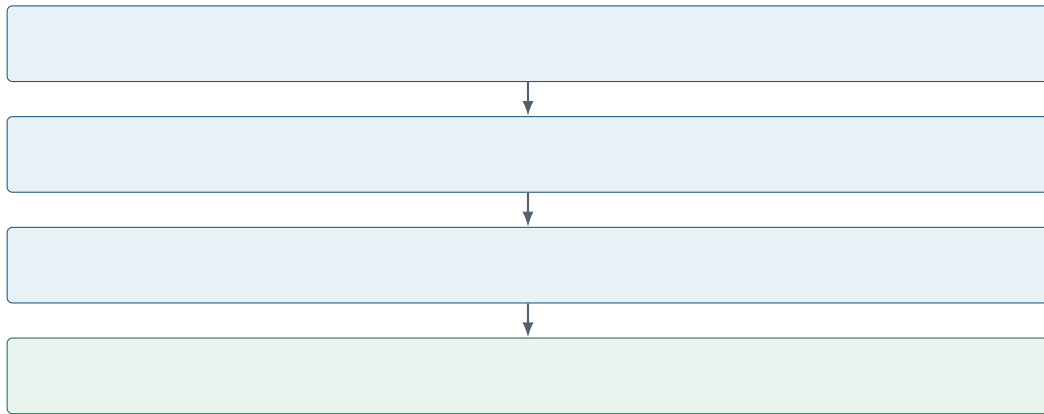
SemanticInventoryName

```
data ProviderSummary = ProviderSummary
  { summaryProvider :: Name
  , summaryScheme   :: CheckedSemanticScheme
  , summaryClauses  :: [SummaryClause]
  , summaryTrust     :: SummaryTrust
  , summaryOrigin   :: SummaryOrigin
  }

data SummaryTrust
  = DerivedFromDatatypeDeclaration
  | LeanKernelChecked TheoremName
  | BuiltinAxiom SemanticModelVersion
  | UserAssumed SourceLocation
  | ObservedOnly Bounds
```

$\text{DataTypeDeclaration}(\text{Cons } x \ xs) = 1 + (xs)$

Declaration



```
Language.Haskell.Synthesis.TypedGenerated
Language.Haskell.Synthesis.Semantic.Sort
Language.Haskell.Synthesis.Semantic.Term
Language.Haskell.Synthesis.Semantic.Formula
Language.Haskell.Synthesis.Semantic.Contract
Language.Haskell.Synthesis.Semantic.Measure
Language.Haskell.Synthesis.Semantic.Summary
Language.Haskell.Synthesis.Semantic.Inventory
Language.Haskell.Synthesis.Semantic.Elaborate
Language.Haskell.Synthesis.Semantic.Symbolic
Language.Haskell.Synthesis.Semantic.Evidence
Language.Haskell.Synthesis.Semantic.CacheKey
```

```
Language.Haskell.Synthesis.Z3.Session
Language.Haskell.Synthesis.Z3.SExpr
Language.Haskell.Synthesis.Z3.SMTLib
Language.Haskell.Synthesis.Z3.Encode
Language.Haskell.Synthesis.Z3.Model
Language.Haskell.Synthesis.Z3.Optimize
Language.Haskell.Synthesis.Z3.Fixedpoint
Language.Haskell.Synthesis.Z3.Verify
```

```
Leant.Synth.Contract
Leant.Synth.Semantic
Leant.Synth.ProofPlan
Leant.Z3
```

Expression

```
data TypedExpression ty local = TypedExpression
  { typedExpressionType :: !ty
  , typedExpressionNode :: TypedNode ty local
  }

data TypedNode ty local
```

```

= TLocal local
| TGlobal Name [VisibleTypeArgument]
| TLambda [(Pattern local, ty)] (TypedExpression ty local)
| TApply (TypedExpression ty local) (TypedExpression ty local)
| TTuple [TypedExpression ty local]
| THole HoleId ty
| TLet (Pattern local) (TypedExpression ty local)
    (TypedExpression ty local)
| TCase (TypedExpression ty local)
    [(TypedPattern ty local, TypedExpression ty local)]

```

```
eraseTypes :: TypedExpression ty local -> Expression local
```

ElaboratedCandidate

```

data SemanticVerdict
  = ContractProved ProofBasis
  | ContractRefuted Counterexample
  | ContractValidatedWithin Bounds ValidationSummary
  | ContractUnknown UnknownReason

data ProofBasis
  = LeanKernelChecked TheoremName
  | Z3Unsat SolverFingerprint EncodingFingerprint
  | CertificateChecked CertificateFingerprint
  | ExactNormalization SemanticModelVersion

data ConstrainedCandidate candidate = ConstrainedCandidate
  { constrainedCandidate :: candidate
  , constrainedVerdict   :: SemanticVerdict
  , constrainedCost      :: SemanticCost
  }

data ConstrainedResult metadata candidate = ConstrainedResult
  { typeSearchResult :: QueryResult metadata candidate
  , semanticResults  :: [ConstrainedCandidate candidate]
  , semanticProgress :: SolverProgress
  }

```

unsat

unsatsatunknown

```
filterConstrained
  :: SolverSession
  -> CheckedContract
  -> SemanticInventory
  -> [QueryResult metadata candidate]
  -> IO (Maybe Progress, ConstrainedAccumulator candidate)
filterConstrained solver contract inventory =
  foldAllQueryResultsM admissible inspect emptyAccumulator
  where
    admissible = candidateHasNoUnsupportedResiduals

    inspect acc candidate = do
      typed <- requireTypedCandidate candidate
      case symbolicInterpret inventory contract typed of
        Left unsupported ->
          pure (recordUnknown candidate unsupported acc)
        Right interpretation -> do
          verdict <- verifyCounterexampleQuery solver contract interpretation
          pure (record candidate verdict acc)
```

foldAllQueryResultsM

k

n

$$h_i : \tau_i e_{i,0}, \dots, e_{i,k_i-1} s_i 0 \leq s_i < k_i$$

$$\llbracket h_i \rrbracket = (s_i = 0, \llbracket e_{i,0} \rrbracket, (s_i = 1, \llbracket e_{i,1} \rrbracket, \dots)).$$

```
cegis :: Sketch -> CheckedContract -> IO SynthesisOutcome
cegis sketch contract = loop initialExamples
  where
    loop examples = do
      model <- solveSelectors sketch contract examples
      case model of
        Unsat -> pure NoProgramInGrammar
        Unknown why -> pure (SynthesisUnknown why)
        Sat assignment -> do
          let candidate = decodeSketch sketch assignment
              counterexample <- verifyCandidate contract candidate
          case counterexample of
            Unsat -> pure (Found candidate)
            Sat input -> loop (input : examples)
            Unknown why -> pure (CandidateUnknown candidate why)
```

```
data SearchExpansion node candidate
  = Solved candidate
  | Expanded [node]
  | DeadEnd

stepExference :: SearchNode -> SearchExpansion SearchNode Candidate

class Monad m => SemanticOracle m node where
  classifyNode :: CheckedContract -> node -> m BranchVerdict

data BranchVerdict
  = DefinitelyInfeasible UnsatExplanation
  | PossiblyFeasible SemanticLowerBound
  | OracleUnknown UnknownReason
```

stepExferenceDefinitelyInfeasible
 $\{n, 0, n + 1\}$

(, , , ,) .

unknown

$F(xs,ys)(xs,ys)$ Badfalse

```
f :: [a] -> [a]
-- Contract: for every admitted xs,
--           length (f xs) = length xs
```

xs

seq

$n = (xs)n \geq 0L_f(n)$

$n \geq 0 \wedge L_f(n) \neq n$

$\backslash xs \rightarrow xs$ n

$\backslash_ \rightarrow []$	0	$n = 1$
tail	$(n = 0, 0, n - 1)$	$n = 1$
$\backslash xs \rightarrow xs ++ xs$	$2n$	$n = 1$
reverse	n	
drop k	$(n - k, 0)$	$k > 0 n = 1 k = 0$
take k	(k, n)	$n = k + 1 k$
$\backslash (x:xs) \rightarrow xs ++ [x]$	n	

$2n$

```
(set-logic QF_LIA)
(set-option :produce-models true)
(declare-const n Int)
(assert (>= n 0))
(define-fun outLen () Int (* 2 n))
(assert (not (= outLen n)))
(check-sat)
(get-value (n outLen))
```

$n = 1 \text{outLen} = 2n \neq \text{nunsat}$

$xs :: [a]$

$$E_{[a]} ::= xs \mid [] \mid (E_{[a]}) \mid E_{[a]} ++ E_{[a]} \\ \mid (E_{[a]}) \mid (E_{\mathbb{N}}, E_{[a]}) \mid (E_{\mathbb{N}}, E_{[a]}) \\ E_{\mathbb{N}} ::= 0 \mid 1 \mid n \mid E_{\mathbb{N}} + E_{\mathbb{N}}.$$

$nn \geq 0$

$$\exists xs. f(xs) \neq xs,$$

$$(f(xs)) = (xs) \quad \wedge \quad (f(xs)) = (xs).$$

seq

```
:synth (f : List alpha -> List alpha)
  where forall xs, (f xs).length = xs.length
```

```
it1 : List alpha -> List alpha := fun xs => xs
```

```
itl_spec : forall xs, (itl xs).length = xs.length := by
  intro xs
  rfl
```

simpomegaunsat

```
data ConstrainedRequest ty options = ConstrainedRequest
  { constrainedTypeRequest :: QueryRequest ty options
  , constrainedContract    :: CheckedContract
  , constrainedPolicy      :: ConstrainedPolicy
  }

data ConstrainedPolicy = ConstrainedPolicy
  { semanticUnknownPolicy :: UnknownPolicy
  , maximumSolverChecks   :: Maybe Natural
  , requiredProofGrade    :: ProofGrade
  , semanticSelection     :: SemanticSelection
  }

runDjinnConstrained
  :: Z3Session
  -> SemanticInventory
  -> DjinnSession
  -> ConstrainedRequest DjinnType QueryOptions
  -> IO (Either Diagnostic DjinnConstrainedResult)

runExferenceConstrained
  :: Z3Session
  -> SemanticInventory
  -> ExferenceSession
  -> ConstrainedRequest ExferenceType ExferenceOptions
  -> IO (Either Diagnostic [ExferenceConstrainedResult])
```

```
verifyCandidate
  :: MonadSemanticSolver m
  => CheckedSemanticContext
  -> TypedCandidate
  -> m SemanticVerdict

foldConstrainedResultsM
  :: Monad m
  => (candidate -> m SemanticVerdict)
  -> ConstrainedFoldPolicy
  -> [QueryResult metadata candidate]
  -> m (ConstrainedSelection candidate)
```

MonadSemanticSolver

Right

```
lengthPreserving ? f :: [a] -> [a]
  where forall xs. length (f xs) == length xs
```

```
:contract f :: [a] -> [a]
  ensures length result == length arg0
:solve f
```

length

Z3_interrupt

$$\exists x. \llbracket f(x) \rrbracket \neq \llbracket g(x) \rrbracket.$$

fg

NatInt

```
@[leant_synth_summary]
theorem List.reverse_length (xs : List alpha) :
  (List.reverse xs).length = xs.length := by
  simp
```

```
SynthCandidates [[String]] [String]
```

```
data SynthCandidate = SynthCandidate
{ synthExpression  :: Expression String
, synthTypedView   :: Maybe LeantTypedCandidate
, synthVariants    :: [String]
, synthOrigin      :: SynthEngine
, synthMetrics     :: CandidateMetrics
}

data SynthOutcome
= SynthCandidates [SynthCandidate] [String]
| SynthRefuted RefutationStrength
| SynthNoTerm [String]
```

sat

it

```
set_option autoImplicit true in
noncomputable example :
  let f : TargetType := candidate
  Contract f := by
    proof_plan
```

```
rflsimp [candidate, summaries] <;> omega
```

rfl

simp

omega

```
:synth (f : List alpha -> List alpha)
  where forall xs, (f xs).length = xs.length

:set synth-contract-proof kernel|solver|bounded
:set synth-semantic-domain length,bag
:set synth-z3-timeout 1500
:set synth-counterexamples on
:set synth-semantic-dedupe on
```

```
1. fun xs => xs
   contract: proved by Lean kernel (rfl)

rejected: fun _ => []
   counterexample: input length = 1, output length = 0

search note: Exference queue pruned 247 nodes
solver note: 8 candidates checked, 5 counterexamples, 2 semantic duplicates
```

k

$$(f, p) \quad p : \forall x, P(x) \rightarrow Q(x, f(x)).$$

```
it1      : TargetType := ...  
it1_spec : Contract it1 := by ...
```

example

```
conflicting clauses:  
  C2: output length = input length  
  C5: output length = input length + 1  
  
or, under the selected grammar:  
  G3: direct argument use disabled  
  G7: reverse provider disabled  
  C1: output is a permutation of input
```

fg

$$((x)) = x$$

get :: S -> A
put :: S -> A
S -> A -> S

$$\begin{aligned} &: \quad (s,(s)) = s, \\ &: \quad ((s,a)) = a, \\ &: \quad ((s,a_1),a_2) = (s,a_2). \end{aligned}$$

$$\text{Word32} \rightarrow \text{Word32}$$

$$L_{out} = aL_{in} + b$$

$$\text{:laws term}$$

$$0 \leq i < (xs) \quad \Rightarrow \quad ((xs,i,v),i) = v.$$

$$(f(xs)) = (xs)$$

xs

$fseq$

f

MathematicalTotal
FiniteSpineTotal
BoundedOperational

```
data SemanticCapability
  = TotalStructural
  | MayInspectPayloadStrictly
  | MayDiverge
  | MayThrow
  | UsesEffects
  | UsesUnsafeFeatures
  | UsesTypeReflection
```

$$\gamma(\llbracket e \rrbracket) \supseteq \{\llbracket e \rrbracket(v) \mid v \in \gamma(input)\}.$$

unknown

unsat

process

```
z3 -in -smt2
```

SemanticSolver

echo

```
(push 1)
(assert (! (>= n 0) :named pre_nonnegative))
(assert (! (not (= outLen n)) :named violates_length))
(check-sat)
(get-info :reason-unknown)
(echo "|djex-request-000042-end|")
(pop 1)
```

get-modelsatget-unsat-coreunsat

Z3_interrupt

```
|d42.list_length|
|p17.Data.List.reverse|
|h3.result|
|c5.contract_clause_length_preserved|
```

unknown

$[a] \rightarrow [a]$

$(a,b) \rightarrow (b,a)$

SynthOutcome

rflsimpomega

itN_spec

Djex/synthesis/.../
Generated.hs
Djex/synthesis/.../
Candidate.hs
Djex/synthesis/.../
Selection.hs
Djex/synthesis/.../Query. QueryEvidence
hs
Djex/djinn/.../
ProofToGenerated.hs
Djex/exference/.../
Internal/Candidate.hs
Djex/exference/.../
ExferenceNode.hs
Djex/synthesis/.../
Semantic/*
Djex/z3/...
Leant/src/Leant/Synth/
Fragment.hs
Leant/src/Leant/Synth/
Engine.hs
Leant/src/Main.hs
Leant/src/Leant/Synth/
ProofPlan.hs

unsatunsat

unsafePerformIO
reversesortlength

```

data Sort
  = SBool
  | SInt
  | SBitVector Natural
  | SUninterpreted SemanticTypeId
  | SDatatype SemanticDatatypeId [Sort]
  | SSequence Sort
  | SArray Sort Sort
  deriving (Eq, Ord, Show)

data Term variable
  = TVar variable Sort
  | TBool Bool
  | TInt Integer
  | TBV Natural Integer
  | TApp FunctionId [Term variable] Sort
  | TItc (Formula variable) (Term variable) (Term variable)
  | TConstructor ConstructorId [Term variable] Sort
  | TSelector SelectorId (Term variable) Sort
  | TSeqEmpty Sort
  | TSeqUnit (Term variable)
  | TSeqConcat [Term variable]
  | TSelect (Term variable) (Term variable) Sort
  | TStore (Term variable) (Term variable) (Term variable)
  deriving (Eq, Ord, Show)

data Formula variable
  = FTrue
  | FFalse
  | FNot (Formula variable)
  | FAnd [Formula variable]
  | FOr [Formula variable]
  | FImplies (Formula variable) (Formula variable)
  | FEq (Term variable) (Term variable)
  | FLt (Term variable) (Term variable)
  | FLe (Term variable) (Term variable)
  | FPredicate PredicateId [Term variable]
  deriving (Eq, Ord, Show)

```

```

data ContractSource ty name = ContractSource
  { sourceInputs      :: [(name, ty)]
  , sourceResultName  :: name
  , sourcePrecondition :: SourceFormula name
  , sourcePostcondition :: SourceFormula name
  , sourceDomain      :: SemanticDomain
  , sourceObjectives  :: [Objective]
  }

data ContractError ty name
  = DuplicateContractBinder name
  | UnboundContractVariable name
  | ContractSortMismatch Sort Sort
  | ContractResultTypeMismatch ty ty
  | UnknownMeasure Name

```

```
| MeasureNotDefinedAtType Name ty
| UnsupportedContractOperator SourceSpan
| DomainIncompatibleProvider Name SemanticCapability
```

```
mkCheckedContract
  :: CheckedSemanticEnvironment ty
  -> QueryRequest ty options
  -> ContractSource ty name
  -> Either (ContractError ty name) CheckedContract
```

CheckedContract

```
data SymbolicValue variable
  = ScalarValue (Term variable)
  | DatatypeValue SemanticDatatypeId [Sort] (Term variable)
  | AbstractValue (Map ObservationId (Term variable))
```

```
data InterpretationError
  = UnsupportedExpression NodePath String
  | MissingProviderSummary Name ObservationId
  | SummaryTypeMismatch Name
  | DomainViolation SemanticCapability
  | RecursiveCandidateRequiresFixedpoint
  | ResidualConstraintUnsupported String
```

```
interpretCandidate
  :: CheckedSemanticEnvironment ty
  -> CheckedContract
  -> TypedExpression ty local
  -> Either InterpretationError (SymbolicFunction local)
```

AbstractValue

```
data Counterexample = Counterexample
  { counterexampleInputs    :: Map BinderId SemanticValue
  , counterexampleResult    :: Maybe SemanticValue
  , violatedClauses         :: [ContractClauseId]
  , counterexampleMinimality :: MinimalityStatus
  , rawModelFingerprint     :: Fingerprint
  }
```

```
data UnknownReason
  = SolverReturnedUnknown Text
  | SolverTimedOut Duration
  | SolverInterrupted
  | SolverProtocolFailure Text
  | UnsupportedSemantics InterpretationError
  | ProofReconstructionFailed Text
```

```
data SolverProgress = SolverProgress
  { solverChecks      :: Natural
  , solverSat         :: Natural
  , solverUnsat       :: Natural
  , solverUnknown     :: Natural
  , solverCacheHits   :: Natural
  , solverElapsed     :: NominalDiffTime
  }
```

```

data Sketch ty local = Sketch
{ sketchRoot      :: TypedExpression ty local
, sketchHoles     :: Map HoleId (HoleAlternatives ty local)
, sketchCostModel :: CostModel
, sketchGrammarKey :: GrammarFingerprint
}

data HoleAlternatives ty local = HoleAlternatives
{ holeType      :: ty
, holeScope     :: CheckedScope ty local
, holeChoices   :: NonEmpty (TypedExpression ty local)
}

data SynthesisOutcome candidate
= SynthesisFound candidate SemanticVerdict
| SynthesisGrammarUnsat GrammarExhaustion UnsatExplanation
| SynthesisUnknown UnknownReason
| SynthesisCancelled

```

```

(set-logic QF_LIA)
(set-option :produce-models true)

; 0 = id, 1 = empty, 2 = duplicate, 3 = tail0
(declare-const choice Int)
(assert (and (<= 0 choice) (< choice 4)))

(define-fun out-len ((n Int)) Int
  (ite (= choice 0) n
    (ite (= choice 1) 0
      (ite (= choice 2) (* 2 n)
        (ite (= n 0) 0 (- n 1))))))

; Current CEGIS examples.
(assert (= (out-len 0) 0))
(assert (= (out-len 1) 1))
(assert (= (out-len 2) 2))

(check-sat)
(get-value (choice))

```

choice = 0

```

(set-logic QF_LIA)
(set-option :produce-models true)

(declare-const n Int)
(assert (>= n 0))

; Decoded choice 0: identity.
(define-fun out-len () Int n)

```

```
(assert (not (= out-len n)))
```

```
(check-sat)
; unsat: no finite nonnegative input length violates the law.
```

```
(assert (not (and (= choice0 2) (= choice1 0) (= choice2 4))))
```

```
set_option autoImplicit true in
noncomputable example :
  let f : List alpha -> List alpha := fun xs => xs
  forall xs, (f xs).length = xs.length := by
  intro xs
  rfl
```

```
set_option autoImplicit true in
noncomputable example :
  let f : List alpha -> List alpha := List.reverse
  forall xs, (f xs).length = xs.length := by
  intro xs
  simp [f]
```

```
set_option autoImplicit true in
noncomputable example :
  let f : List alpha -> List alpha := fun xs =>
    match xs with
    | [] => []
    | x :: xt => xt ++ [x]
  forall xs, (f xs).length = xs.length := by
  intro xs
  cases xs with
  | nil => rfl
  | cons x xt =>
    simp [f]
```

```
set_option autoImplicit true in
noncomputable example :
  let f : Nat -> Nat := fun n => (n + 1) - 1
  forall n, f n = n := by
  intro n
  simp [f]
```

```
simp [f, registered_summary_1, registered_summary_2]
```

omega

simpomega

rfl
decide
omega

Djex/README.md
Djex/synthesis/README.md
Djex/.../Synthesis/Query.hs
Djex/.../Synthesis/Search.hs
Djex/.../Synthesis/Candidate.hs
Djex/.../Synthesis/Generated.hs
Djex/.../Synthesis/Selection.hs
Djex/.../Synthesis/Declaration.hs
Djex/.../Djex/Djinn.hs
Djex/.../Djex/Exference.hs
Djex/.../Internal/ExferenceNode.hs
Djex/.../Internal/Exference.hs
Djex/djex.cabal

Leant/README.md
Leant/src/Leant/Synth/Engine.hs
Leant/src/Leant/Synth/Fragment.hs
Leant/src/Main.hs
Leant/leant.cabal

```
z3/README.md
z3/src/api/z3_api.h
z3/src/api/z3_optimization.h
z3/src/api/z3_fixedpoint.h
z3/src/shell/main.cpp
z3/src/api/api_seq.cpp
```

```
unsat
```

```
sorry
```